



Shri Vile Parle Kelavani Mandal's

DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING

(Autonomous College Affiliated to the University of Mumbai)

NAAC Accredited with "A" Grade (CGPA : 3.18)



Mini Project on "Project Title"

SUBMITTED IN PARTIAL FULFILMENT OF THE REQUIREMENTS
OF THE DEGREE OF

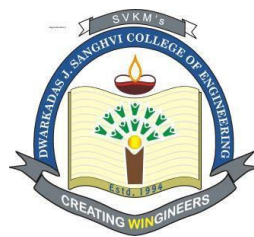
**Bachelor of Technology in Computer Science and Engineering
(IoT and Cyber Security with Blockchain Technology)**

by

Dhriti Vaidya	B023
Gargi Sukhatankar	B027
Meenakshi Chandra	B039

Project Guide Name

Prof. Rohini Bhosale



**Department of Computer Science and Engineering
(IoT and Cyber Security with Block chain Technology)**



Shri Vile Parle Kelavani Mandal's

DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING

(Autonomous College Affiliated to the University of Mumbai)

NAAC Accredited with "A" Grade (CGPA : 3.18)



SVKM's Dwarkadas Jivanlal Sanghvi College of Engineering
(Autonomous College Affiliated to the University of Mumbai)
No.U-15, J.V.P.D. Scheme, Bhaktivedanta Swami Marg, Opp.Cooper Hospital,
Vile Parle (West), Mumbai-400 056. India

Academic Year 2024-2025

Project Stage-I

Music Recommendation System

Submitted in Partial fulfillment of the requirements of, Cloud Computing Laboratory (DJS22ICL604) (Sem-VI) in the Department of Computer Science & Engineering (IoT and Cyber security with Block Chain Technology)

by

Dhriti Vaidya	B023
Gargi Sukhatankar	B027
Meenakshi Chandra	B039

Project Guide Signature with Name and Date:

Prof. _____

Date: _____



Shri Vile Parle Kelavani Mandal's

DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING

(Autonomous College Affiliated to the University of Mumbai)

NAAC Accredited with "A" Grade (CGPA : 3.18)



Department of Computer Science and Engineering
(IoT and Cybersecurity with Block Chain Technology)

Certificate

This is to certify that, topic entitled “**Music Recommendation System** ” has been reviewed and evaluated by undersigned members, and is submitted as partial fulfillment of Cloud Computing Laboratory (DJS22ICL604) (Sem-VI) in the Department of Computer Science and Engineering (IoT and Cybersecurity with Block Chain Technology)

External Examiner Signature:

External Examiner Name :

Internal Examiner Signature:

Internal Examiner Name :

Dr. Narendra Shekokar
(Vice Principal & Head of Department)

Dr. Hari Vasudevan
(Principal)



Abstract

With the overwhelming volume of music on digital platforms, users often struggle to discover songs that truly resonate with their personal tastes. This project aims to develop a Music Recommender System that provides personalized song recommendations by analyzing user preferences, listening history, and song metadata.

The core of the system will utilize machine learning algorithms, such as content-based filtering, to learn from user behavior and suggest music aligned with individual tastes.

Cloud Storage will be utilized for managing and hosting large datasets, including audio features, user logs, and metadata. This hybrid approach ensures efficient and secure data management without the need for deploying the entire system on the cloud.

By separating storage concerns to the cloud and maintaining compute operations locally, the project balances scalability and simplicity. The recommender system will remain dynamic, scalable in terms of data, and responsive to user needs, without depending on full cloud deployment. This setup also makes it easier to scale in the future if full cloud hosting becomes a requirement.



Table of Contents

Chapter	Contents	Page No.
1	Introduction	6
2	Project Description	7
3	Implementation Details	8
4	Results/Analysis	16
5	Conclusion	18



Introduction

With the explosion of digital content in the music industry, users are constantly faced with an overwhelming amount of options across streaming platforms. Discovering songs that match individual preferences has become a challenge, especially when traditional recommendation engines rely heavily on generic metrics like genre or popularity. In this context, cloud computing has emerged as a critical enabler for building intelligent, scalable, and efficient recommendation systems. Cloud platforms such as Amazon Web Services (AWS) offer a range of services for high-capacity data storage, secure access, and on-demand computing resources. These capabilities are essential for managing the vast volumes of song metadata, user interaction logs, and lyrical content that power modern recommendation algorithms.

This project emphasizes the storage and computational power of the cloud, particularly through the use of AWS S3 buckets, which allow for efficient storage and retrieval of large-scale datasets. By offloading heavy data operations to the cloud, the system achieves improved performance, scalability, and accessibility, while keeping the local compute infrastructure lightweight. The focus on lyric-based recommendations further adds complexity to data processing, which is efficiently handled through cloud-based preprocessing and streaming techniques. Ultimately, cloud computing in this project is not just a support system but a core foundation for the architecture, enabling real-time data access, flexible model development, and seamless user interaction.



Project Description

This project, titled "A Cloud-Based Music Recommendation System Using Lyrics Analysis," aims to create a personalized music discovery experience by analyzing the lyrical content of songs. Unlike traditional systems that rely on genre classifications or user listening patterns, this system uses content-based filtering to find songs with similar lyrical themes. At its core, the system integrates Natural Language Processing (NLP) techniques—such as tokenization, TF-IDF vectorization, and cosine similarity—to evaluate lyrical features and compute song similarity scores. The uniqueness of this project lies in its cloud-native design, where all song data is securely stored and managed on Amazon S3, ensuring high availability and scalability.

The dataset used is the Spotify Million Song Dataset, which includes rich song metadata, user interaction data, and full song lyrics. Due to its size, storing this dataset locally would limit performance and access. Instead, it is hosted on AWS S3, where it can be accessed on demand by the recommendation engine. The system uses Python libraries such as `pandas`, `scikit-learn`, `boto3`, and `nltk` to preprocess the lyrics and generate recommendations dynamically. A user-friendly Streamlit-based UI allows users to enter a favorite song or keywords and receive real-time, thematically similar song suggestions, along with relevant metadata and optional previews.

In its current architecture, the project runs on local machines but connects to the cloud for data retrieval, balancing resource usage while maintaining high responsiveness. In future iterations, the entire system—including the backend logic and UI—can be deployed to the cloud using platforms like Streamlit Cloud, AWS EC2, or Render, turning it into a fully cloud-hosted solution. The project demonstrates how cloud storage, combined with machine learning and NLP, can power an intelligent, scalable, and user-centric music recommendation experience.

Implementation Details

The Music Recommendation System uses Content-Based Filtering to recommend music based on user preferences, leveraging a large dataset stored on AWS Cloud. The system combines machine learning, cloud storage, and a user-friendly UI for a seamless experience.

□ Dataset Preparation and Preprocessing

A large-scale dataset containing song metadata (genre, mood, artist, tempo, etc.) and user interaction data (likes, listens, ratings) was used to train the recommendation model. The dataset utilized is the **Spotify Million Song Dataset**

(<https://www.kaggle.com/datasets/notshrirang/spotify-million-song-dataset>), which provides rich metadata and user interaction information for millions of songs. Due to the dataset's size, it was uploaded to an Amazon S3 bucket for efficient and scalable storage. This setup provides:

- **Scalability:** Handles large datasets without storage limitations.
- **Availability:** Access from any platform, including deployment environments.
- **Security:** Managed access via IAM policies and S3 bucket permissions.

```
df.head(10)
# df.tail(10)
```

	artist	song	link	text
0	ABBA	Ahe's My Kind Of Girl	/a/abba/ahes+my+kind+of+girl_20598417.html	Look at her face, it's a wonderful face \r\nA...
1	ABBA	Andante, Andante	/a/abba/andante+andante_20002708.html	Take it easy with me, please \r\nTouch me gen...
2	ABBA	As Good As New	/a/abba/as+good+as+new_20003033.html	I'll never know why I had to go \r\nWhy I had...
3	ABBA	Bang	/a/abba/bang_20598415.html	Making somebody happy is a question of give an...
4	ABBA	Bang-A-Boomerang	/a/abba/bang+a+boomerang_20002668.html	Making somebody happy is a question of give an...
5	ABBA	Burning My Bridges	/a/abba/burning+my+bridges_20003011.html	Well, you hoot and you holler and you make me ...
6	ABBA	Cassandra	/a/abba/cassandra_20002811.html	Down in the street they're all singing and sho...
7	ABBA	Chiquitita	/a/abba/chiquitita_20002978.html	Chiquitita, tell me what's wrong \r\nYou're e...
8	ABBA	Crazy World	/a/abba/crazy+world_20003013.html	I was out with the morning sun \r\nCouldn't s...
9	ABBA	Crying Over You	/a/abba/crying+over+you_20177611.html	I'm waitin' for you baby \r\nI'm sitting all ...

```
df = df.sample(5000).drop('link', axis=1).reset_index(drop=True)
```

+ Code + Text

df

	artist	song	text
0	Mariah Carey	Shake It Off	[Chorus] \r\nI gotta shake you off \r\n'Caus...
1	Chaka Khan	Everybody Has An Aura	Yeah, uh hmm \r\nYeah, uh hmm \r\nYeah, uh h...
2	Queens Of The Stone Age	Make It Wit Chu	You wanna know if I know why \r\nI can't say ...
3	Amy Grant	Baby Baby	Baby, baby \r\nI'm taken with the notion \r\...
4	Cake	Haze Of Love	It's three o'clock in the morning \r\nOr mayb...
...	...	...	...
4995	Opeth	Weakness	Found you there in the blink of an eye \r\nI ...
4996	Oasis	Rock 'n' Roll Star	I live my life in the city \r\nThere's no eas...
4997	INXS	Doctor	We're all running fast tonight \r\nRocket sho...
4998	Elvis Presley	Clambake	All right, Hmm \r\nMamma's little baby loves ...
4999	Creedence Clearwater Revival	Get Down Woman	Well, get down woman, before I have to go. \r...

5000 rows x 4 columns

□ Model Development

This isn't your average genre-matching engine. The system analyzes song **lyrics** to recommend similar tracks using:

- **Text Vectorization:** Lyrics are preprocessed (lowercased, tokenized, stop words removed) and transformed using **TF-IDF** or **word embeddings**.
- **User Preference Modeling:** User selects a favorite song or keyword(s), and the system captures the underlying lyrical themes.
- **Similarity Matching:** Songs with similar lyrical content are found using **cosine similarity** between TF-IDF vectors.

Libraries Used:

- pandas, NumPy – classic data sidekicks.
- scikit-learn – for TF-IDF vectorization and similarity metrics.
- nltk or spaCy – for NLP preprocessing.
- boto3 – for downloading the dataset from AWS S3.

```
import nltk
nltk.download('punkt_tab')
from nltk.stem.porter import PorterStemmer
from nltk.tokenize import word_tokenize
```

```
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data] Package punkt_tab is already up-to-date!
```

```
[ ] stemmer = PorterStemmer()
```

```
[ ] def token(txt):
    token = nltk.word_tokenize(txt)
    a = [stemmer.stem(w) for w in token]
    return " ".join(a)
```

```
def token(txt):
    token = nltk.word_tokenize(txt)
    a = [stemmer.stem(w) for w in token]
    return " ".join(a)
```

[+ Code](#)

```
df['text'].apply(lambda x: token(x))
```

```
[ ]
```

	text
0	[choru] i got ta shake you off 'caus the lov...
1	yeah , uh hmm yeah , uh hmm yeah , uh hmm stra...
2	you wan na know if i know whi i ca n't say tha...
3	babi , babi i 'm taken with the notion to love...
4	it 's three o'clock in the morn or mayb it 's ...
...	...
4995	found you there in the blink of an eye i miss ...
4996	i live my life in the citi there 's no easi wa...
4997	we 're all run fast tonight rocket shoe on our...

```
[ ] from sklearn.feature_extraction.text import TfidfVectorizer
    from sklearn.metrics.pairwise import cosine_similarity

[ ] tfidf = TfidfVectorizer(analyzer='word', stop_words='english')

[ ] matrix = tfidf.fit_transform(df['text'])

[ ] similar = cosine_similarity(matrix)

[ ] similar[0]

⇒ array([1.          , 0.00607543, 0.01303795, ..., 0.00987609, 0.02484233,
         0.          ])

[ ] int(df[df['song'] == 'Magic Mirror'].index[0])

⇒ 0
```

```
[ ] #Recommender system
    def recommender(song_name):
        idx = int(df[df['song'] == song_name].index[0])
        distance = sorted(list(enumerate(similar[idx])), reverse=True, key=lambda x:x[1])
        song = []
        for s_id in distance[1:11]:
            song.append(df.iloc[s_id[0]].song)
        return song
```

```
recommender("Magic Mirror")
```

```
['The Magic Mirror',  
'Dance In The Mirror',  
'Stranger In My Mirror',  
'This Is My Night',  
'Magic',  
'I'll Be Your Mirror',  
'Go To The Mirror',  
'Only You',  
'Fairy Tale Lullaby',  
'Preservation']
```

```
[ ] import pickle
```

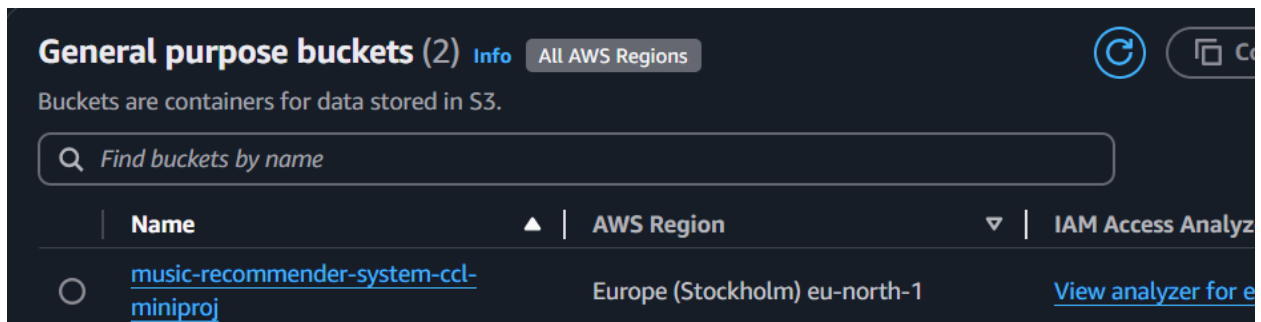
```
[ ] pickle.dump(similar, open("similarity", "wb"))
```

```
[ ] pickle.dump(similar, open("df", "wb"))
```

□ Integration with AWS S3

Given the size of the dataset, AWS S3 was used as the cloud-based storage solution:

- **Access Control:** The IAM user was granted s3:GetObject permissions to securely fetch the dataset.
- **Access Keys:** Keys were stored securely in environment variables to prevent leakage.
- **Data Loading:** The system fetches the dataset from S3 on startup and loads it into memory for real-time processing.



General purpose buckets (2) Info All AWS Regions

Buckets are containers for data stored in S3.

Find buckets by name

Name	AWS Region	IAM Access Analyz
music-recommender-system-ccl-miniproj	Europe (Stockholm) eu-north-1	View analyzer for e

music-recommender-system-ccl-miniproj Info

Objects**Properties****Permissions****Metrics****Manage****Objects (1)**

Copy S3 URI



Copy URL



Do

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon](#) need to explicitly grant them permissions. [Learn more](#)

**Name****Type**[spotify_millsongdata.csv](#)

csv



```
from dotenv import load_dotenv
load_dotenv(".env")
```

```
import os
```



```
[ ] DATASET_URI = os.getenv('DATASET_URI')
```



```
#pulling data from amazon aws bucket.
df = pd.read_csv(DATASET_URI)
```

□ Streamlit UI for User Interaction

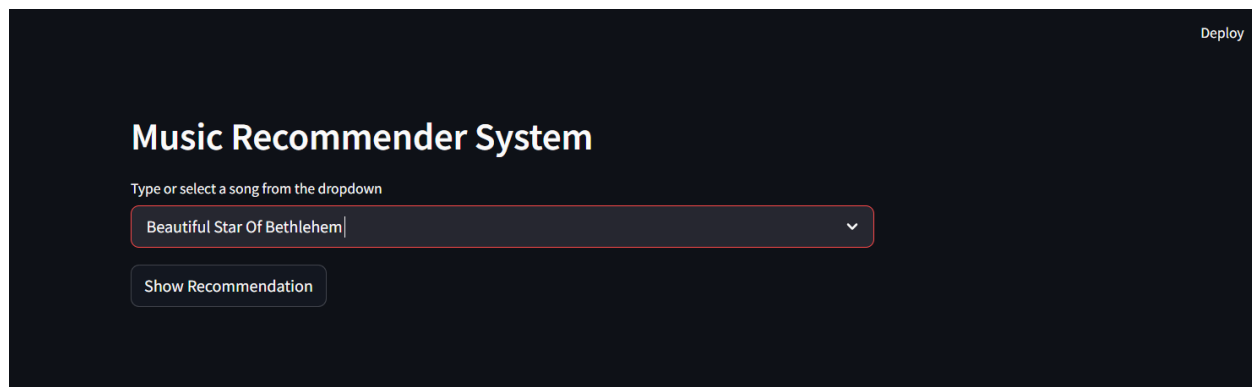
The user interface for the system is built using Streamlit, offering an easy-to-use platform for users to interact with the recommendation system:

- **Song Input:** Users can input song preferences (e.g., favorite genres or artists).
- **Real-Time Recommendations:** Based on input, Streamlit dynamically displays a list of recommended songs.
- **Song Information:** Each recommendation includes metadata (artist, genre) and a play button for song previews.

The Streamlit app connects to the recommendation system, processes the user input, and retrieves personalized recommendations in real-time. The simple yet effective UI allows users to easily interact with the system and receive song suggestions.

Streamlit Features:

- **Interactive Inputs:** Users can input their preferences via sliders, dropdowns, and text fields.
- **Real-Time Updates:** Song recommendations update instantly based on user interactions.
- **Music Previews:** Song previews (audio or metadata) are available to provide a rich user experience.



Music Recommender System

Type or select a song from the dropdown

Beautiful Star Of Bethlehem

Show Recommendation

Beautiful, Loved
And Blessed



Lost In The Stars



A Cradle In
Bethlehem



Stars Fall From
Heaven



Never Be A Star



Deployment and Future Plans

Currently, the recommendation system runs locally via Streamlit. However, the system is designed to scale for future deployment on cloud platforms such as Render, AWS EC2, or Streamlit Cloud, with S3 continuing as the central data storage solution.

Results/Analysis

1. System Performance

a. Dataset Handling

- The dataset used for this system was the [Spotify Million Song Dataset](#), which contains metadata and lyrics of over a million songs.
- Due to the large size of the dataset, it was uploaded and accessed via **Amazon S3**, which allowed for efficient, scalable, and centralized data handling.

b. Data Preprocessing

- Lyrics were cleaned, tokenized, and vectorized using **TF-IDF** to extract meaningful textual features.
- The preprocessing pipeline performed well on the cloud-hosted dataset with minimal latency during streaming and vectorization.

c. Recommendation Generation

- Once vectorized, lyrics were compared using **cosine similarity** to identify and recommend songs with similar lyrical themes.
- For any input song or lyrics, the top-N most similar songs were generated based on their TF-IDF vectors.

2. Model Output Characteristics

a. Thematic Accuracy

- The model shows promising results in grouping songs with similar lyrical content. Songs discussing themes like love, heartbreak, motivation, or freedom are often recommended alongside others with the same themes, regardless of genre.

b. Genre Independence

- Since the recommendations are purely lyric-based, they can span across different musical genres. For example, a pop song and a country song with similar lyrical sentiments may appear in the same recommendation group.

c. Cold Start Capability



- The system does not rely on user history or collaborative data, making it capable of functioning effectively even with a first-time user — as long as a reference song or lyrics are provided.
-

3. Cloud Performance

a. S3 Integration

- The dataset is stored and accessed through Amazon S3, which allows for fast, secure, and scalable data handling.
- By using **boto3**, data is fetched directly from S3 without requiring local storage, keeping the system lightweight and cloud-native.

b. Load Time & Efficiency

- Initial data load from S3 to the app is efficient due to streaming-based loading and memory-optimized processing.
- Once loaded, data is cached for reuse to avoid repeated calls to the cloud, improving responsiveness.

c. Access Control & Security

- IAM roles and bucket policies were implemented to ensure secure yet functional access.
- Access credentials are stored in environment variables, preventing leakage and maintaining system integrity.

d. Scalability Potential

- The cloud-based infrastructure allows for easy scaling, whether deploying the system on **Streamlit Cloud, Heroku, or AWS EC2**.
 - Additional services (like AWS Lambda or API Gateway) could be integrated in future iterations for serverless architecture and faster API calls.
-

4. Streamlit UI Observations

- **Responsiveness:** The UI built with Streamlit loads quickly and processes inputs in real-time, retrieving and displaying recommendations efficiently.
- **User Inputs:** The application allows users to enter a song name or lyrics and receive recommendations based on lyric similarity.
- **Interactivity:** While the UI is intuitive and functional, it can be further improved with additional filters (e.g., mood, genre, language) and advanced visualizations.



Conclusion

This project demonstrates the successful implementation of a lyric-based music recommendation system using content-based filtering. By focusing on the semantic and thematic essence of song lyrics, the system offers a unique and genre-independent recommendation experience. Leveraging the extensive Spotify Million Song Dataset and integrating cloud storage through AWS S3 enabled scalable data handling and efficient model performance.

The combination of natural language processing, cloud-based architecture, and an interactive Streamlit UI results in a system that is not only technically robust but also user-friendly and ready for real-world application. While the system has not yet undergone large-scale testing, its architecture lays a solid foundation for future enhancements, including deeper NLP models, user profiling, and deployment to scalable platforms.

In summary, the project showcases how machine learning, cloud computing, and intuitive interfaces can come together to redefine music discovery through the lens of lyrical content.
